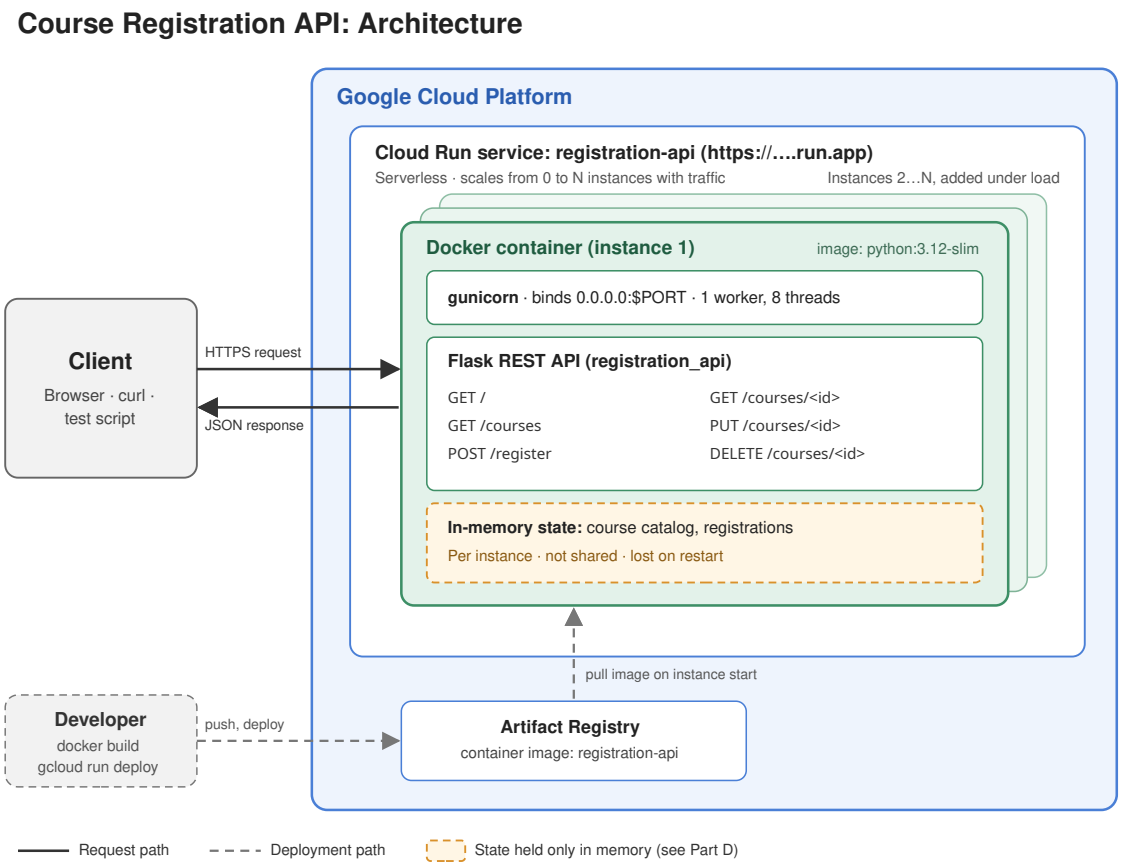
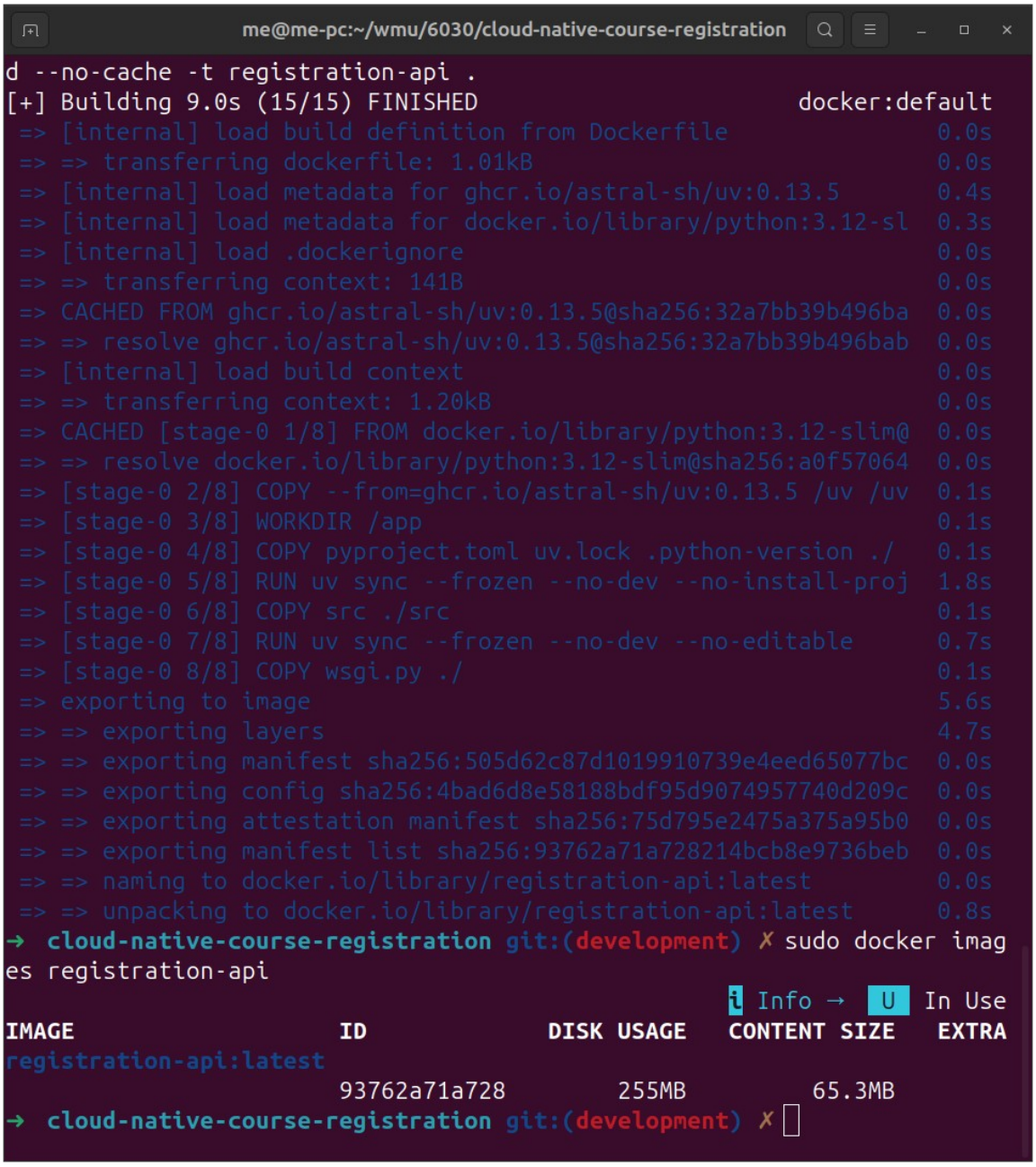
Theodore Podewil
Dr. Ajay Gupta
CS6030 Cloud Computing
Assignment 2
October 24, 2026

A. Architecture Design
   1. Diagram

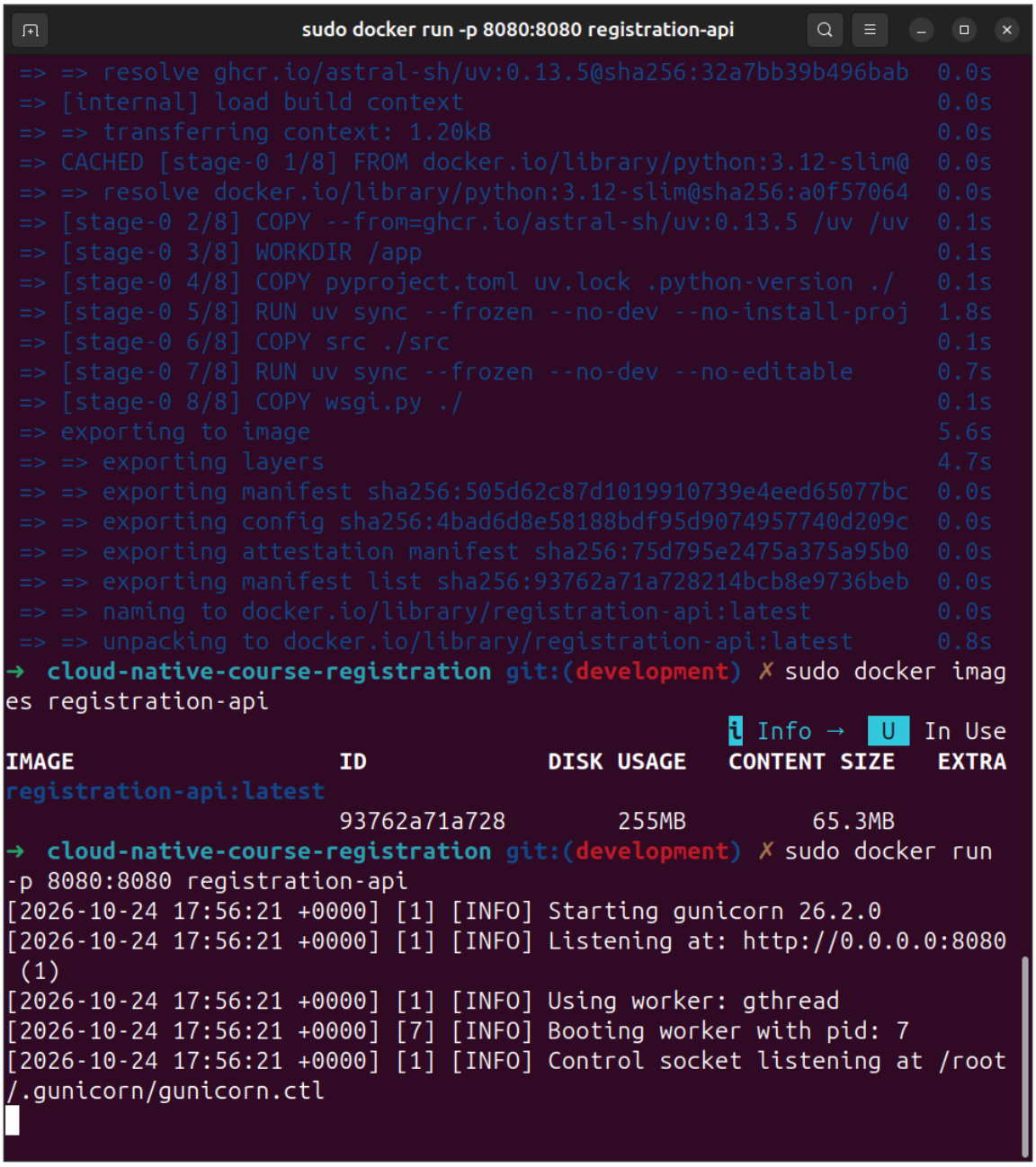


   2. Questions
      a. Why is this architecture considered cloud-native?
         1. The architectures is considered cloud-native because it is packaged as a container, it runs on google cloud run, its configuration comes from the environment, and it scales horizontally.
      b. Which components are stateless?
         1. All of the components are stateless EXCEPT the list of classes registered. The POST requests save a course to an in memory list which is unique to each instance.
      c. How would the system scale?
         1. There are two ways the system scales: within an instance/container and across instances/containers. Gunicorn allows multiple threads within a container. Cloud Run autoscaling will create more instances when the “concurrency” limit is reached (when instances reach the max simultaneous requests).

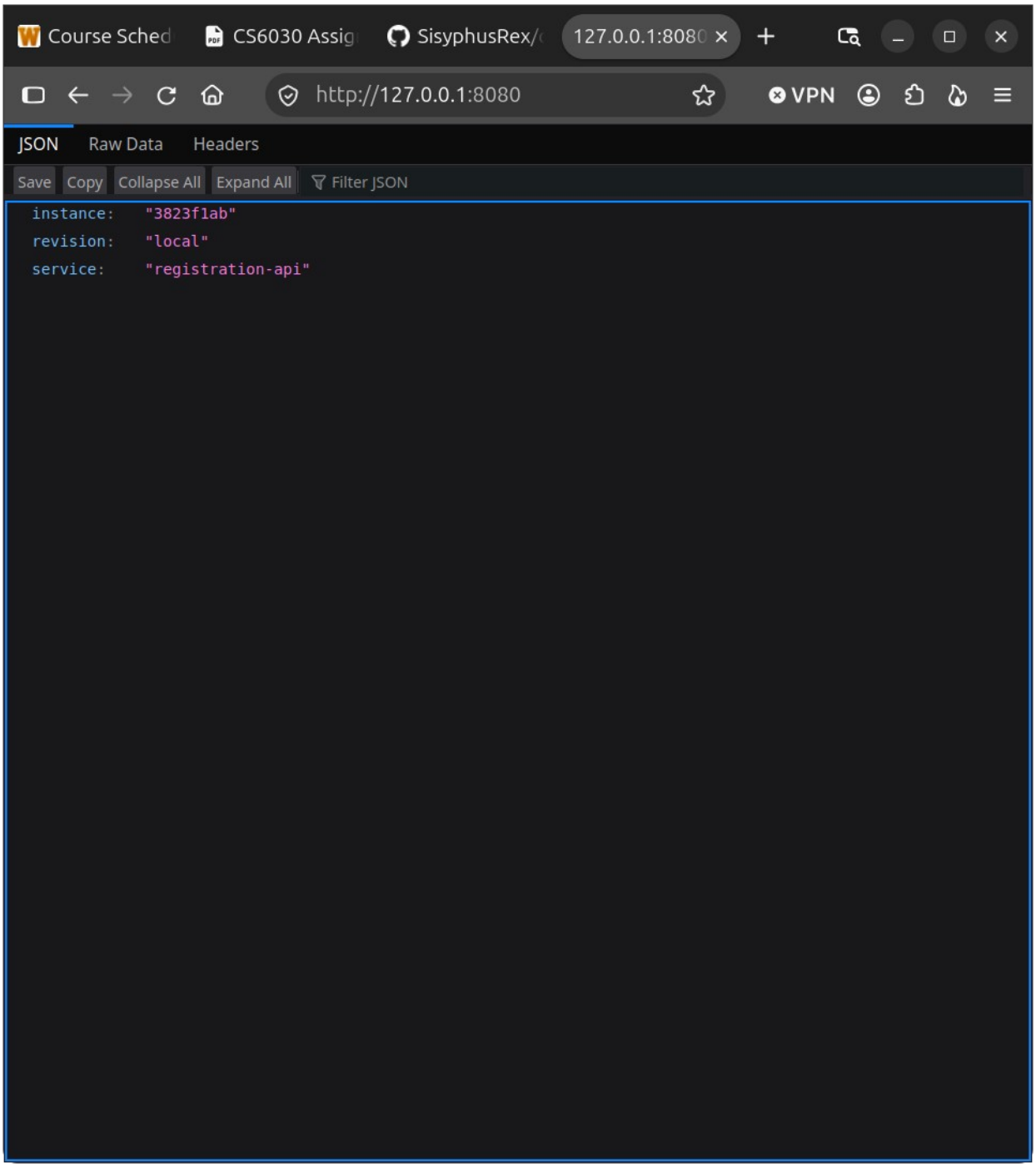
B. Containerization
   1. Build Success



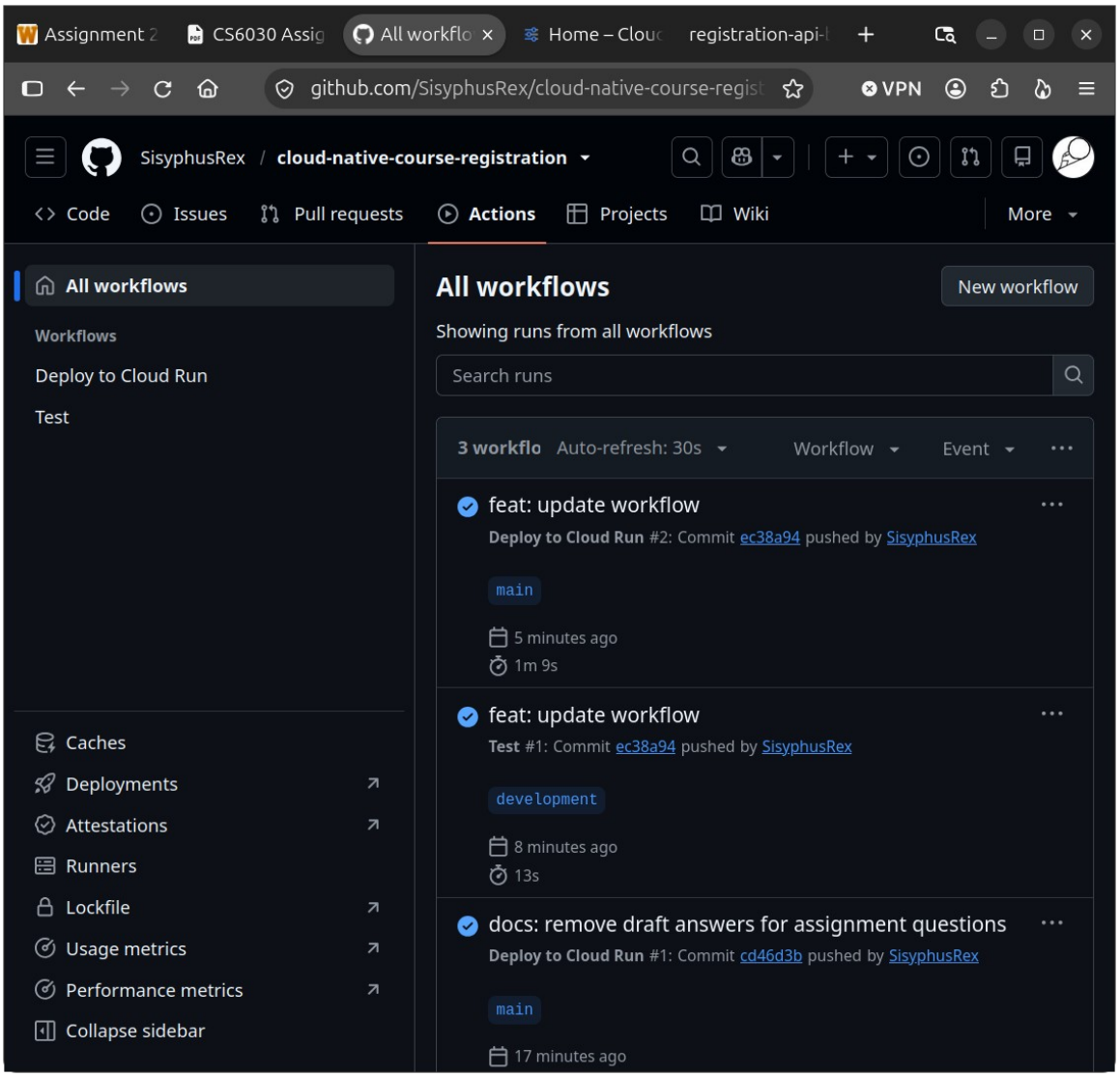
2. Running Container



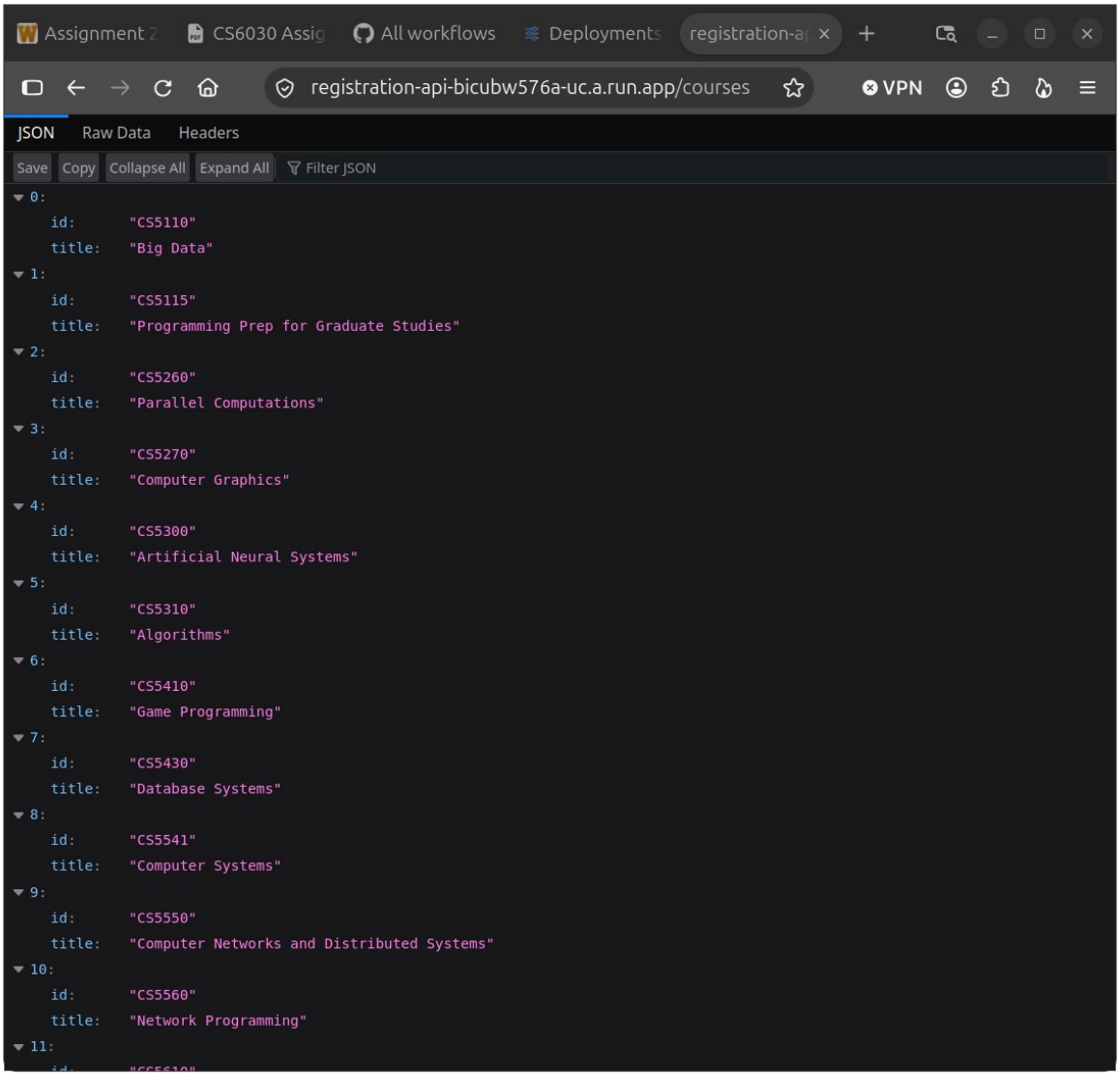
3. API accessible


Course Sched
CS6030 Assig
SisyphusRex/
127.0.0.1:8080
http://127.0.0.1:8080
VPN
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
instance: "3823f1ab"
revision: "local"
service: "registration-api"

C. Deploy to Google Cloud run
   1. Cloud Run service url
      a. https://registration-api-bicubw576a-uc.a.run.app/
   2. Deployment



   3. API Response



D. Cloud Native Evaluation
   1. Is configuration separated from code?
      a. Yes. Deployment settings exist in the github workflow and secrets exist in the github environment. The PORT passed to gunicorn is an environment variable.
   2. Is the service stateless?
      a. No. The registrations list can change between instances. It exists only on the gunicorn process running in that container.
   3. Could multiple copies run simultaneously?
      a. Yes. There are no conflicts between instances and it is set to scale horizontally.
   4. Does it store user state locally?
      a. Yes. The registration POST request appends to a container specific in memory list. That list exists on a gunicorn process unique to each container.

E. VM versus Container Analysis
   1. Would you deploy this on a VM or a container platform?
      a. I would deploy this on a container platform. A container platform allows us to scale up to accommodate the heavy registration load and then scale down for normal load. A VM would require more resources all of the time to handle the one-time heavy registration load.
   2. What advantages does a container provide?

a. A container is fast to start, can scale out to handle large loads and then scale to zero to reduce costs, and has less maintenance than a VM.

3. How would scaling differ?
   a. The biggest differences between scaling a VM and containers are speed and scale to zero. A VM scales to a bigger machine and requires stopping the service, whereas containers just create more containers. A VM must run continuously, which increases cost, while containers can scale to zero.

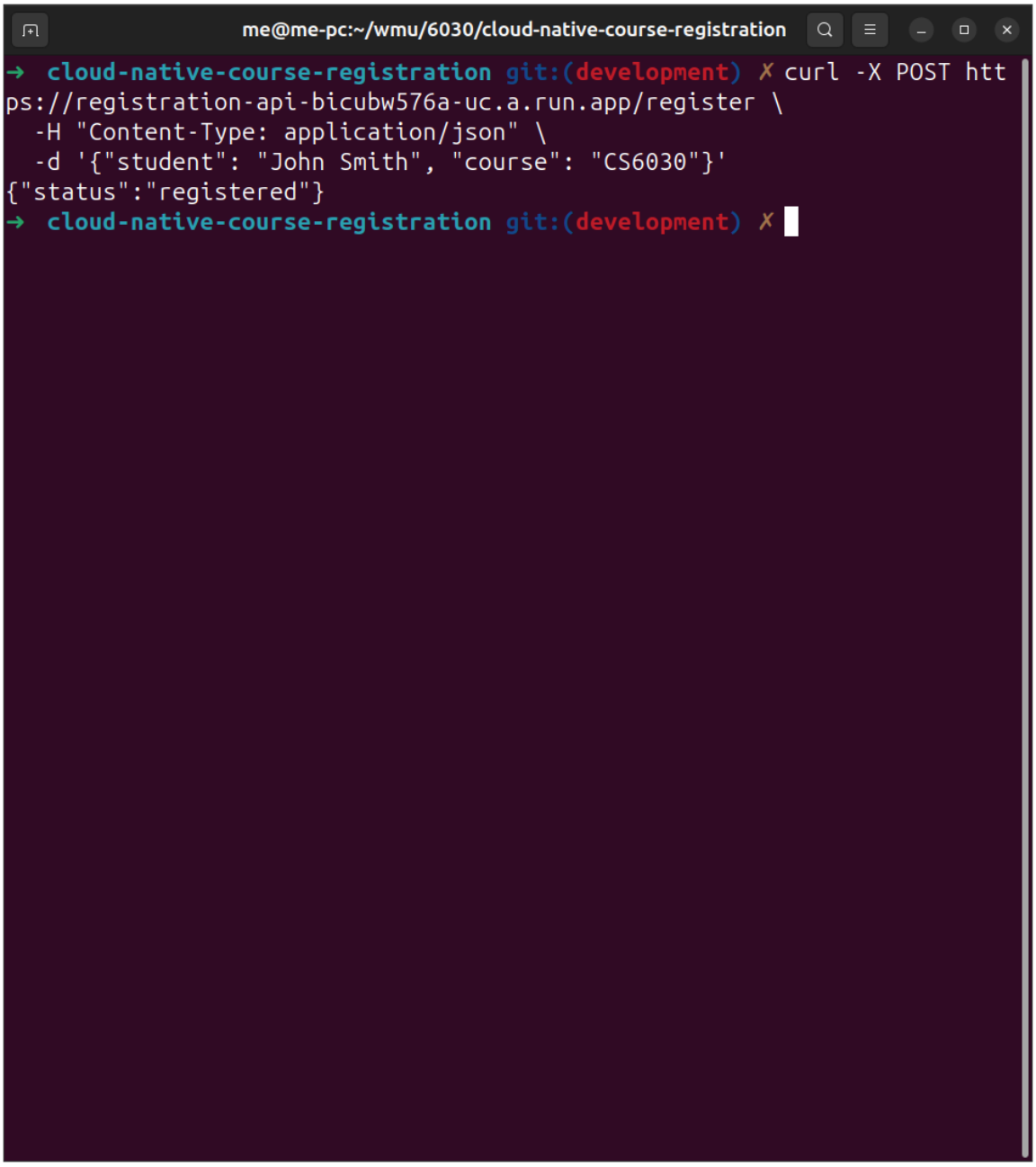
F. Testing and Resilience

1. GET



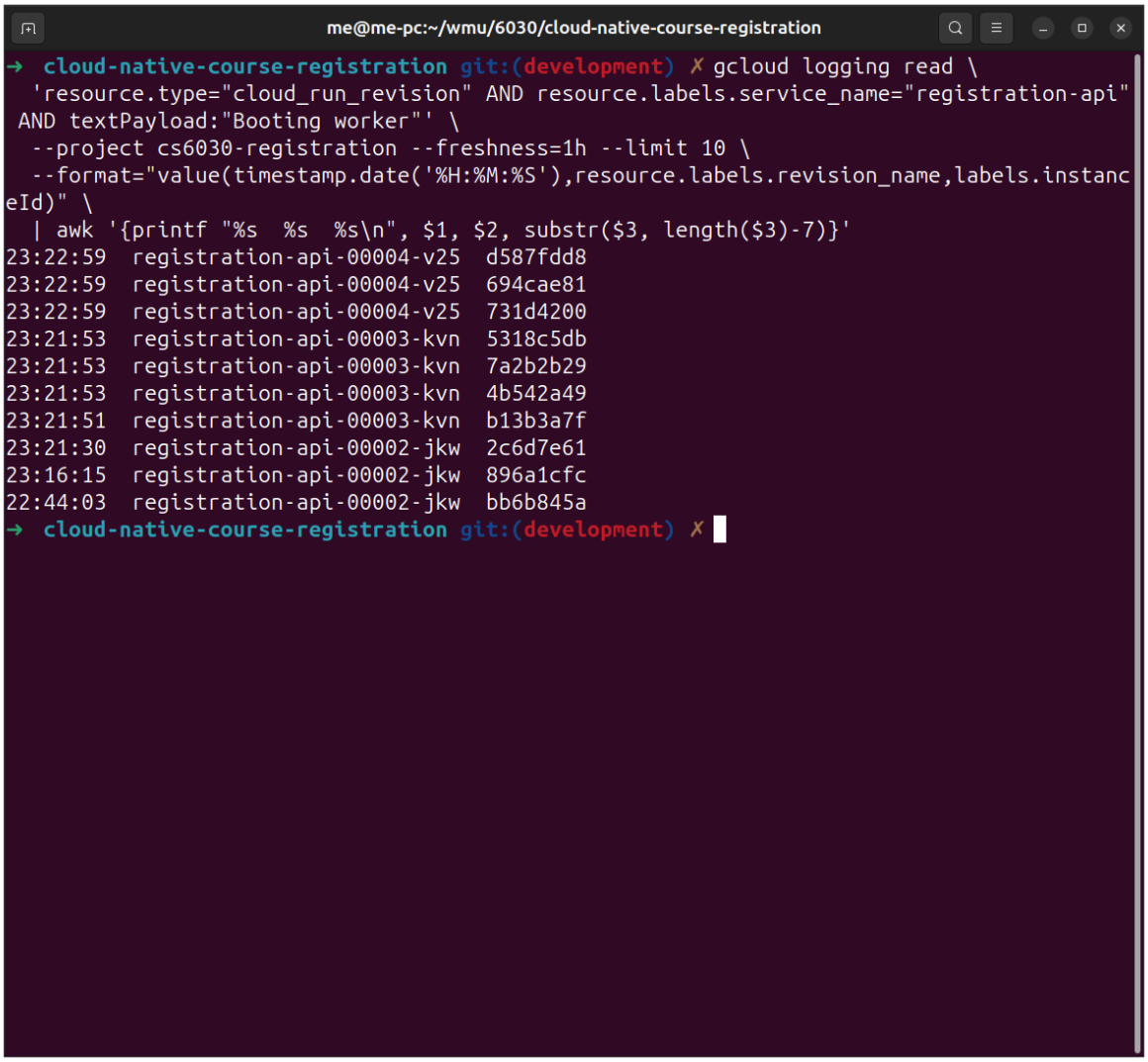
2.

3. POST



4.

5. Multiple Instances



6. Could multiple instances of your application serve these requests?
    a. Yes. I examined gcloud's logging and found 3 instances that spun up at the same time I made my requests.

G. AI Usage

1. Local development server port (`wsgi.py`)

AI Tool(s) Used	Claude Code (Claude Opus 5.5)
Purpose	Understand why `app.run()` read the port from the `PORT` environment variable.
Sample Prompts	"in wsgi.py, why do we overcomplicate app.run() when we can just pass the port number, 8080, directly?" "What sets the port env variable in the dockerfile?"
Generated Code Used	None directly. I made the suggested edit to `wsgi.py` myself.
Verification Performed	Compared the explanation against the `Dockerfile` and gunicorn.

2. Blueprint structure and class-based views

AI Tool(s) Used	Claude Code (Claude Opus 5.5)

Purpose	Organize URL prefixes and convert the blueprints to Flask `MethodView` classes.
Sample Prompts	"Move the prefixes to app.py." "Can you reorganize the blueprints using MethodView? If the blueprint does not have all of the methods (get, post, put, delete) create stubs for them."
Generated Code Used	Yes. The `url_prefix` arguments in `app.py` and the `IndexAPI`, `CoursesAPI` and `RegistrationAPI` classes with 501 stubs.
Verification Performed	Checked output against online resources

3. Single-course resource and tests

AI Tool(s) Used	Claude Code (Claude Opus 5.5)
Purpose	Add GET, PUT and DELETE for `/courses/<course_id>`, with tests.
Sample Prompts	"Don't implement the plan, just show me how courses.py would look." "Can you add the tests?"
Generated Code Used	Yes. The `CourseAPI` class and `find_course()` in `courses.py`, and 10 tests plus a fixture that resets the catalog in `tests/test_app.py`.
Verification Performed	Checked that methods worked.

4. Flask concepts (explanations only)

AI Tool(s) Used	Claude Code (Claude Opus 5.5)
Purpose	Learn how Flask routing and class-based views work.
Sample Prompts	"What does add_url_rule do in the blueprints?" "Why do we need as_view again?"
Generated Code Used	Example snippets only, not added to the project. A summary table was saved as `docs/architecture.md`.
Verification Performed	Checked explanations against online resources

5. Architecture diagram (Part A)

AI Tool(s) Used	Claude Code (Claude Opus 5.5)
Purpose	Create the diagram showing the client, REST API, Docker container and cloud platform.
Sample Prompts	"Could you create an architecture diagram showing the Client, REST API, Docker Container, and Cloud Platform per the ASSIGNMENT.md?" "How did the https endpoint get into the diagram?"
Generated Code Used	Yes. `docs/architecture-diagram.svg` and its PNG render.
Verification	Compared the diagram against assignment requirements.

Performed	

8. CI/CD: deploy only on merges to `main`

AI Tool(s) Used	Claude Code (Claude Opus 5.5)
Purpose	Run the tests on every change and deploy to Cloud Run only when code reaches `main`.
Sample Prompts	"How could we set up this project to only deploy when merged to main?" "I've created the environment, can you create the github files"
Generated Code Used	Yes. `.github/workflows/test.yml` and `.github/workflows/deploy.yml`. I created the GitHub `production` environment myself.
Verification Performed	Merged to main and looked to see if deployment occurred.

9. Google Cloud project and keyless authentication

AI Tool(s) Used	Claude Code (Claude Opus 5.5)
Purpose	Create a separate GCP project and let GitHub Actions deploy without a stored service account key (Workload Identity Federation).
Sample Prompts	"Do i need to create a project on google cloud to set the region?" "can you run the gcloud commands?"
Generated Code Used	`gcloud` commands to enable the APIs and create the deploy service account, the identity pool and provider, and their bindings. I ran them myself, except the final fix, which the AI ran at my request. I added the GitHub secrets myself.
Verification Performed	Checked to see if project was created on gcloud

10. Branch ruleset for `main`

AI Tool(s) Used	Claude Code (Claude Opus 5.5)
Purpose	Require passing tests on `main` while still allowing direct fast-forward pushes.
Sample Prompts	"how do i make a ruleset for main that requires tests to pass but allows merging to main and pushing." "add checks gives me test and test/test as options"
Generated Code Used	Yes. A `development` push trigger in `test.yml`, which I committed myself. I created the ruleset myself in GitHub, following the AI's steps.
Verification Performed	Created the rulesets.

11. Deploy pipeline diagram

AI Tool(s) Used	Claude Code (Claude Opus 5.5)
Purpose	Show each step from a push to `main` to the code running on Cloud Run.
Sample Prompts	"can you make a diagram of the steps from pushing to main to the code running on cloud run?" "commit and push"
Generated Code Used	Yes. `docs/deploy-pipeline.svg` and its PNG render.
Verification Performed	I reviewed the diagrams.

14. Part F testing scripts

AI Tool(s) Used	Claude Code (Claude Opus 5.5)
Purpose	Run Tests 1 to 3 and watch Cloud Run add instances under load.
Sample Prompts	"Could you write a script to test the get and post requests in part F of the assigment?" "Can you make the script make enough requests to force gcloud to create multiple instances and monitor the instances being created?"
Generated Code Used	Yes. `scripts/part_f_tests.sh` (GET and POST tests) and `scripts/part_f_load_test.py` (simultaneous requests that report each new instance).
Verification Performed	The AI ran both scripts against the live service. All GET and POST checks passed. With concurrency lowered to 1, three instances started within 3 seconds and handled 75,391 requests with no errors, which Cloud Monitoring and the service logs confirmed. The script restored the original setting afterwards.